Pensamiento computacional

Nombre: _____ Fecha: _____

Qué vas a aprender

- Entender qué es el pensamiento computacional y para qué sirve
- Aplicar sus cuatro pilares: descomposición, patrones, abstracción y algoritmos
- Resolver problemas cotidianos y de programación paso a paso
- Depurar soluciones buscando y corrigiendo errores

Para pensar

¿Qué tienen en común organizar una fiesta de cumpleaños, resolver un cubo de Rubik y programar un videojuego?

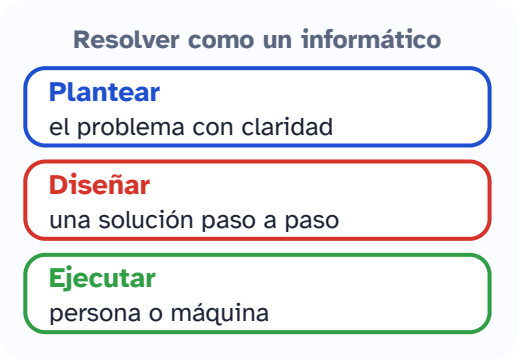
Explicación

1.1 ¿Qué es el pensamiento computacional?

El **pensamiento computacional** es una forma de resolver problemas inspirada en cómo trabajan los informáticos: plantear el problema de manera que una persona o una máquina pueda ejecutar la solución.

No consiste en pensar como un ordenador, sino en organizar nuestras ideas para que un ordenador nos ayude. La investigadora **Jeannette Wing** lo popularizó en 2006 y defendió que debía ser una habilidad básica, como leer, escribir o calcular.

Se apoya en **cuatro pilares**: descomposición, reconocimiento de patrones, abstracción y algoritmos.



1.2 Los cuatro pilares

Pilar	Qué significa	Ejemplo: preparar una mochila de excursión
Descomposición	dividir un problema grande en partes pequeñas	comida, ropa, botiquín, documentos
Patrones	buscar semejanzas o repeticiones	cada día necesito lo mismo: repetir la lista
Abstracción	quedarse con lo esencial e ignorar lo irrelevante	importa el peso y el tiempo, no el color de la mochila
Algoritmo	escribir los pasos ordenados de la	1) mirar el tiempo, 2) elegir ropa, 3) meter

Pilar	Qué significa	Ejemplo: preparar una mochila de excursión
	solución	comida…

Los cuatro se usan juntos y en cualquier orden.



2.1 Descomposición

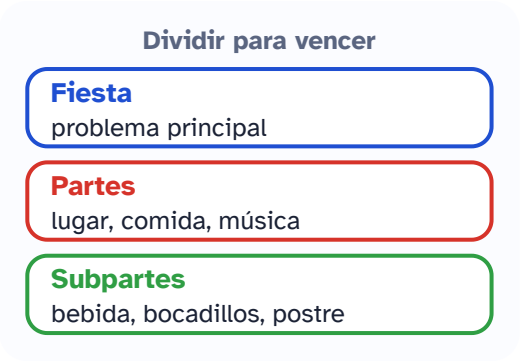
Un problema grande asusta; muchos problemas pequeños, no. **Descomponer** es dividir un problema en subproblemas más sencillos que se pueden resolver por separado (y a veces por personas distintas).

Por ejemplo, programar un videojuego de plataformas se descompone en:

- mover al personaje con el teclado,
- hacer que salte y caiga (gravedad),
- detectar las plataformas,
- contar puntos y vidas,
- mostrar la pantalla de fin de partida.

Cada parte puede volver a dividirse hasta que sea fácil de programar. Así trabajan los equipos de software: cada persona se encarga de un módulo.

Una buena forma de descomponer es dibujar un **árbol**: el problema principal arriba y, debajo, sus partes, que a su vez se dividen en otras más pequeñas, hasta llegar a tareas que se saben resolver directamente. Por ejemplo, «hacer una fiesta de fin de curso» se divide en lugar, comida, música y decoración; «comida» se divide en bebida, bocadillos y postre; y así sucesivamente. Además, las partes independientes se pueden repartir entre varias personas y hacerse a la vez.



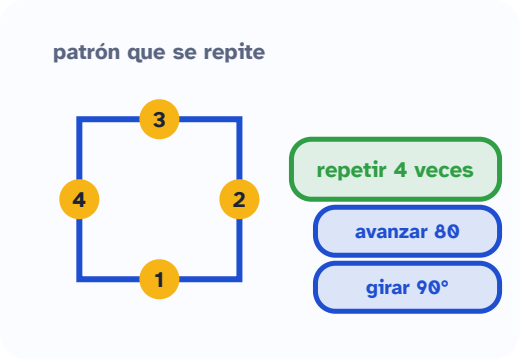
2.2 Reconocimiento de patrones

Un **patrón** es algo que se repite. Detectarlo ahorra trabajo: si ya sabemos resolver un caso, podemos reutilizar la solución en los parecidos.

- En matemáticas: 2, 4, 6, 8… cada número es el anterior más 2.
- En un dibujo: un cuadrado se dibuja repitiendo 4 veces «avanza y gira 90°».
- En un juego: todos los enemigos se mueven igual, solo cambia su posición inicial.

En programación, los patrones se convierten en `bucles` (repetir) y en `funciones` (bloques reutilizables). Detectar patrones es lo que permite que un programa corto haga mucho trabajo.

Los patrones también aparecen en los errores: si un programa falla siempre que el número es negativo, el patrón del fallo indica dónde buscar. Y en los datos: las aplicaciones de música detectan qué canciones suelen escuchar juntas las personas con gustos parecidos para recomendarte otras.



2.3 Abstracción

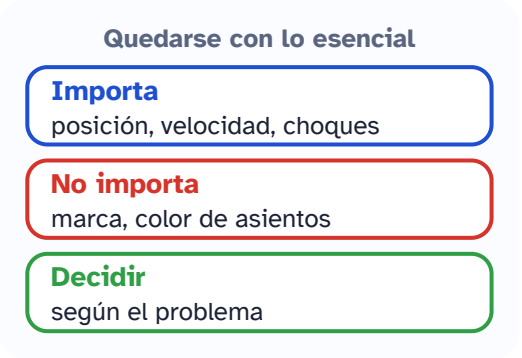
Abstraer es quedarse con la información importante para resolver el problema y descartar los detalles que no influyen. Es una de las ideas más poderosas de la

informática.

El ejemplo clásico es el **plano del metro**: no respeta las distancias reales ni las curvas de las vías. Solo muestra lo que necesita el viajero: estaciones, líneas y conexiones. Por eso es tan fácil de usar.

Al programar un juego de fútbol, no necesitamos simular cada músculo del jugador: basta con su posición, su velocidad y si tiene el balón. Elegir bien qué datos guardar es la base de un buen programa.

Abstraer bien depende del problema. Para un juego de carreras, de un coche importan la posición, la velocidad y si choca; no importan la marca de los neumáticos ni el color de los asientos. Pero en un simulador de conducción profesional, el desgaste de los neumáticos sí importa. Elegir qué detalles se conservan es una decisión de diseño: si se quitan demasiados, la solución no sirve; si se dejan todos, el problema se vuelve inabarcable.

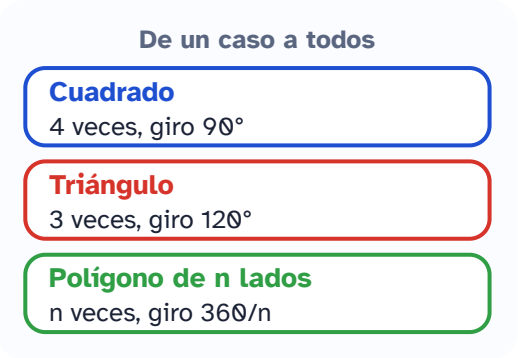


2.4 Generalización: una solución para muchos casos

La **generalización** consiste en convertir la solución de un caso concreto en una solución que sirva para todos los casos parecidos. Une el reconocimiento de patrones y la abstracción. Si sabemos dibujar un cuadrado (repetir 4 veces: avanzar y girar 90°) y un triángulo (repetir 3 veces: avanzar y girar 120°), descubrimos el patrón y generalizamos: para cualquier polígono regular de **n** lados, repetir n veces: avanzar y girar 360/n grados.

En programación, la generalización se consigue con **variables** y **funciones con parámetros**: en lugar de escribir un programa para la tabla del 7 y otro para la del 8, se escribe uno que reciba el número como dato. Una solución general ahorra trabajo, se puede reutilizar y se adapta a casos nuevos sin empezar de cero. La

pregunta clave es: «¿qué cambia de un caso a otro y qué se mantiene igual?». Lo que cambia se convierte en un parámetro.



2.5 Representar la información: todo son datos

Para que un ordenador resuelva un problema, la información del mundo real tiene que convertirse en **datos**, y en último término en números binarios (ceros y unos). Es otra forma de abstracción. Un texto se guarda como una lista de códigos: cada letra tiene un número (en el código ASCII, la «A» es el 65). Una imagen digital es una cuadrícula de **píxeles**, y el color de cada píxel se describe con tres números del 0 al 255: la cantidad de rojo, verde y azul (**RGB**). El blanco es (255, 255, 255) y el negro, (0, 0, 0).

Un sonido se representa midiendo muchas veces por segundo la altura de la onda (44.100 veces por segundo en un CD). Un mapa se puede convertir en un **grafo**: puntos (cruces, ciudades) unidos por líneas con un número (la distancia o el tiempo). Elegir bien cómo representar los datos facilita mucho el algoritmo: el GPS no «ve» las calles, sino un grafo en el que busca el camino de menor peso.



3.1 Algoritmos

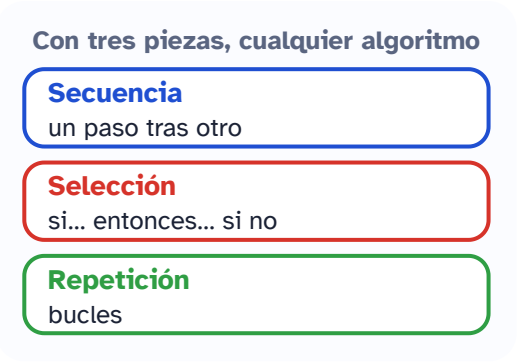
Un **algoritmo** es una secuencia finita, ordenada y precisa de instrucciones para resolver un problema. Una receta de cocina, las instrucciones de un mueble o el camino para llegar a clase son algoritmos.

Para que un ordenador lo ejecute, el algoritmo debe ser:

- **Preciso**: sin ambigüedades. «Echa un poco de sal» no sirve para una máquina; «añade 5 g de sal», sí.
- **Ordenado**: los pasos tienen una secuencia.
- **Finito**: termina en algún momento.
- **Con entrada y salida**: recibe datos y produce un resultado.

En el próximo tema aprenderás a representarlos con diagramas de flujo.

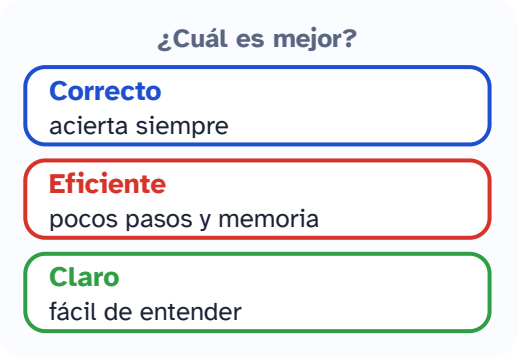
Los algoritmos se construyen con solo tres estructuras básicas, que se combinan entre sí: la **secuencia** (un paso detrás de otro), la **selección** o condicional (elegir un camino según una condición) y la **repetición** o bucle (repetir unos pasos). Se ha demostrado que con estas tres piezas se puede escribir cualquier algoritmo, desde una receta hasta el programa que controla un avión. Antes de programar, conviene expresar el algoritmo en pseudocódigo o con un diagrama de flujo y comprobarlo a mano.



3.2 Evaluar soluciones: ¿cuál es mejor?

Un mismo problema puede resolverse con varios algoritmos, y no todos son igual de buenos. Para compararlos se tiene en cuenta si son **correctos** (dan siempre el resultado adecuado, también en casos extremos), si son **eficientes** (cuántos pasos o cuánto tiempo y memoria necesitan) y si son **claros** (fáciles de entender y de modificar).

Un ejemplo: para encontrar un nombre en una lista ordenada de 1.000 alumnos, se puede mirar uno a uno desde el principio (en el peor caso, 1.000 comparaciones) o abrir por la mitad y descartar la mitad que no sirve cada vez (la **búsqueda binaria**: unas 10 comparaciones). Con un millón de nombres, el primer método puede necesitar un millón de pasos y el segundo, unos 20. Cuando hay muchos datos, la eficiencia marca la diferencia entre una aplicación que responde al instante y otra que tarda horas. Pero a veces la solución más sencilla basta: si la lista tiene 10 nombres, no merece la pena complicarse.



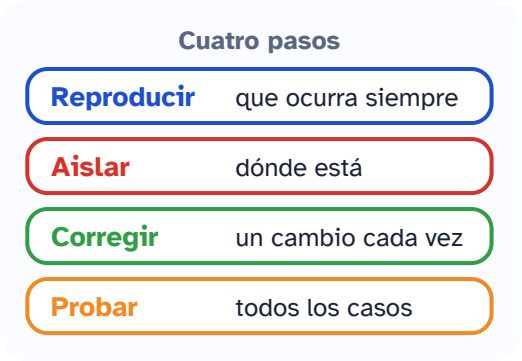
3.3 Depurar: aprender de los errores

Casi ningún programa funciona a la primera. **Depurar** (en inglés, *debugging*) es buscar y corregir los errores, llamados **bugs**. Se cuenta que el término se popularizó cuando la programadora **Grace Hopper** y su equipo encontraron una polilla atascada en un ordenador en 1947.

Estrategias para depurar:

- Ejecutar el programa **paso a paso** y comprobar qué hace en cada momento.
- Probar con datos distintos, incluidos casos extremos (0, números negativos, listas vacías).
- Cambiar **una sola cosa** cada vez.
- Explicar el código en voz alta a otra persona (o a un patito de goma).

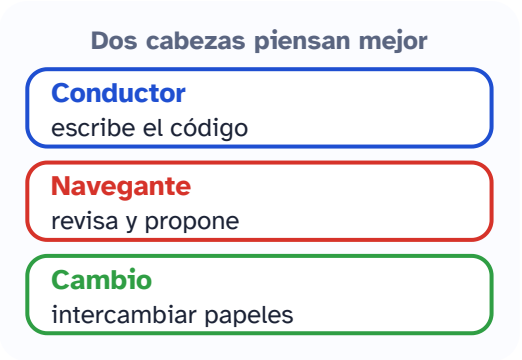
Un buen método de depuración tiene cuatro pasos: **reproducir** el error (conseguir que ocurra siempre), **aislarlo** (encontrar en qué parte del programa está), **corregirlo** cambiando una sola cosa y **volver a probar**, también los casos que antes funcionaban, para no estropear otra parte. Probar con casos extremos (el 0, números negativos, una lista vacía) descubre muchos errores ocultos.



3.4 Resolver problemas en equipo

Los grandes programas los hacen equipos, y el pensamiento computacional también se practica en grupo. En la **programación en parejas** una persona escribe (el **conductor**) y la otra revisa, piensa en el siguiente paso y detecta errores (el **navegante**); cada cierto tiempo intercambian los papeles. Así se cometen menos errores y las dos aprenden.

Para colaborar bien conviene: descomponer el problema y repartir las partes según las habilidades de cada uno; ponerse de acuerdo en los nombres de las variables y en cómo se comunican las partes; explicar el propio razonamiento en voz alta; aceptar las críticas como ayuda y no como ataque; y valorar los errores como una oportunidad de aprender. La **perseverancia** es una actitud clave del pensamiento computacional: los problemas difíciles rara vez se resuelven al primer intento.



4.1 Pensamiento computacional sin ordenador

Estas habilidades se pueden practicar **desenchufadas**, sin pantallas:

- **Robot humano**: un compañero da órdenes precisas («avanza 2 pasos», «gira a la derecha») para que otro llegue a un objetivo con los ojos tapados.

- **Cuadrícula de colores**: codificar un dibujo píxel a píxel con números y que otro lo reconstruya.
- **Ordenar**: buscar el método más rápido para ordenar cartas de menor a mayor.
- **Juegos de lógica**: sudokus, laberintos, el juego del Nim.

Así se entiende que el pensamiento computacional es una manera de pensar, no una tecnología.



4.2 Para qué sirve en la vida real

El pensamiento computacional está detrás de casi todo lo digital: el GPS que calcula la ruta más corta, las recomendaciones de series, la predicción del tiempo o los robots que ayudan en un quirófano.

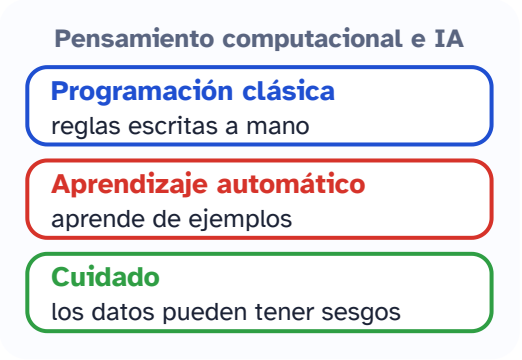
Pero también es útil sin ordenador: organizar el estudio de un examen (descomponer el temario), detectar errores que repites en los ejercicios (patrones), resumir un tema (abstracción) o planificar una tarea en grupo (algoritmo). Por eso se considera una competencia clave del siglo XXI y forma parte de la **competencia digital** del currículo.



4.3 Pensamiento computacional e inteligencia artificial

En la programación clásica, las personas escriben el algoritmo paso a paso. En el **aprendizaje automático**, una rama de la inteligencia artificial, el ordenador construye su propio modelo a partir de muchos **ejemplos**: si se le muestran miles de fotos etiquetadas como «gato» o «perro», aprende a distinguirlos sin que nadie le diga qué es una oreja o un bigote. Herramientas educativas como Teachable Machine o Machine Learning for Kids permiten entrenar un modelo propio con la cámara y usarlo en Scratch.

El pensamiento computacional sigue siendo imprescindible: hay que **descomponer** el problema, decidir qué datos usar (**abstracción**), comprobar si el modelo acierta y **depurarlo** cuando falla. Y hay que pensar en sus límites: si los ejemplos de entrenamiento tienen **sesgos** (por ejemplo, solo fotos de un tipo de personas), el modelo los repetirá. Entender cómo funciona la IA nos ayuda a usarla con criterio y a no creer todo lo que produce.



Los cuatro pilares y la depuración

Descomposición	dividir el problema en partes pequeñas
Patrones	buscar lo que se repite → bucles y funciones
Abstracción	quedarse con lo esencial (plano del metro)
Algoritmo	pasos finitos, ordenados y precisos
Depuración	probar, encontrar el bug y corregirlo

Ejercicios

1. **Define los cuatro pilares del pensamiento computacional.**

2. **Descompón el problema «organizar una excursión de clase».**

3. **Encuentra el patrón: 3, 7, 11, 15… ¿Qué número sigue?**

4. **¿Qué información necesitas para programar un semáforo? ¿Qué puedes ignorar?**

5. **Reescribe de forma precisa: «calienta la leche un rato».**

6. **¿Qué ángulo se gira en cada vértice para dibujar un triángulo equilátero con «avanza y gira»?**

7. **Un programa suma bien 2 + 3 pero falla con 0 + 0. ¿Qué estrategia de depuración se aplicó?**

8. **Explica por qué el plano del metro es una abstracción.**

9. **Escribe el algoritmo para lavarte los dientes en 6 pasos.**

10. **Inventa una actividad desenchufada para enseñar bucles a niños de primaria.**

11. **¿Qué es la generalización? Pon un ejemplo.**

12. **¿Qué tres criterios se usan para comparar algoritmos?**

13. **¿Qué hacen el conductor y el navegante en la programación en parejas?**

14. **¿Cómo se representa el color de un píxel en RGB?**

15. **¿Qué diferencia hay entre programar y entrenar un modelo de IA?**

16. **Dibuja el árbol de descomposición de «preparar una mochila para una excursión».**

17. **Generaliza el algoritmo de un robot que cruza una cuadrícula de 3 × 3 para una cuadrícula de n × n.**

18. **¿Qué blanco, negro y gris medio tienen en RGB?**

19. **Una imagen de 100 × 100 píxeles en RGB usa 3 bytes por píxel. ¿Cuánto ocupa sin comprimir?**

20. **Aplica el método de depuración de cuatro pasos a: «el contador de vidas a veces baja de 0».**

21. **¿Cuántas comparaciones necesita como máximo la búsqueda binaria en 1.000 elementos? ¿Y en 1.000.000?**

22. **Representa como grafo el camino de casa al instituto con tres cruces y las distancias entre ellos.**

23. **¿Por qué la perseverancia es importante en el pensamiento computacional?**

24. **Explica qué es un sesgo en un modelo de IA con un ejemplo.**

25. **Propón una abstracción para una aplicación de horarios de autobús: qué datos guardas y cuáles no.**

26. **Diseña un algoritmo general para repartir n pedidos en una furgoneta usando la estrategia «ir siempre al más cercano». ¿Da siempre la ruta óptima?**

27. **Compara dos algoritmos para encontrar el número mayor de una lista: ordenarla entera y coger el último, o recorrerla una vez guardando el mayor. ¿Cuál es más eficiente y por qué?**

28. **Planifica cómo entrenarías un modelo que reconozca tres gestos con la cámara y lo usarías en Scratch. Aplica los cuatro pilares.**

29. **Una imagen Full HD tiene 1.920 × 1.080 píxeles. ¿Cuánto ocupa sin comprimir en RGB (3 bytes/píxel)? ¿Por qué se comprimen las imágenes?**

30. **Aplica el pensamiento computacional a organizar el estudio de final de trimestre con seis exámenes en dos semanas.**

Practica

1. **Dividir un problema grande en partes pequeñas es...**

   A) abstracción
   B) descomposición
   C) depuración
   D) programación

2. **Quedarse con lo esencial e ignorar detalles irrelevantes es...**

   A) patrón
   B) algoritmo
   C) abstracción
   D) depuración

3. **Buscar lo que se repite corresponde al pilar de...**

   A) reconocimiento de patrones
   B) descomposición
   C) depuración
   D) abstracción

4. **Un algoritmo debe ser...**

   A) largo y complejo
   B) escrito en inglés
   C) finito, ordenado y preciso
   D) secreto y rápido

5. **«Echa un poco de sal» no es una buena instrucción para una máquina porque...**

   A) es muy larga
   B) es ambigua
   C) no tiene números romanos
   D) no usa verbos

6. **Buscar y corregir errores en un programa se llama...**

   A) depurar
   B) compilar
   C) abstraer
   D) ejecutar

7. **¿Quién popularizó el término pensamiento computacional en 2006?**

   A) Grace Hopper
   B) Jeannette Wing
   C) Ada Lovelace
   D) Steve Jobs

8. **Repetir «avanza y gira 90°» cuatro veces dibuja un…**

   A) triángulo
   B) círculo
   C) cuadrado
   D) pentágono

9. **Una actividad desenchufada es la que se hace…**

   A) sin ordenador
   B) sin internet pero con tablet
   C) con robots
   D) con pantallas táctiles

10. **Al depurar, conviene cambiar…**

    A) todo a la vez
    B) una sola cosa cada vez
    C) nada
    D) solo la última

11. **Convertir una solución concreta en una que sirva para muchos casos es…**

    A) generalizar
    B) depurar
    C) descomponer
    D) ejecutar

12. **Para dibujar cualquier polígono de n lados se gira…**

    A) 90°
    B) 360/n grados
    C) n grados
    D) 180°

13. **En RGB, (0, 0, 0) es…**

    A) blanco

B) negro
C) rojo
D) gris

14. **Una imagen digital está formada por...**

A) píxeles
B) átomos
C) grafos
D) bits de sonido

15. **Un mapa representado con puntos y líneas con pesos es un...**

A) píxel
B) grafo
C) bucle
D) sprite

16. **La búsqueda binaria en 1.000 elementos necesita unos...**

A) 1.000 pasos
B) 10 pasos
C) 500 pasos
D) 2 pasos

17. **En programación en parejas, el navegante...**

A) escribe
B) revisa y propone
C) se va
D) solo mira el reloj

18. **Un modelo de aprendizaje automático aprende a partir de...**

A) reglas escritas
B) ejemplos
C) la suerte
D) un único dato

19. **Si los datos de entrenamiento no son variados, el modelo puede tener...**

A) sesgos
B) más velocidad
C) menos memoria

D) colores

20. **El primer paso para depurar es...**

A) borrar todo

B) reproducir el error

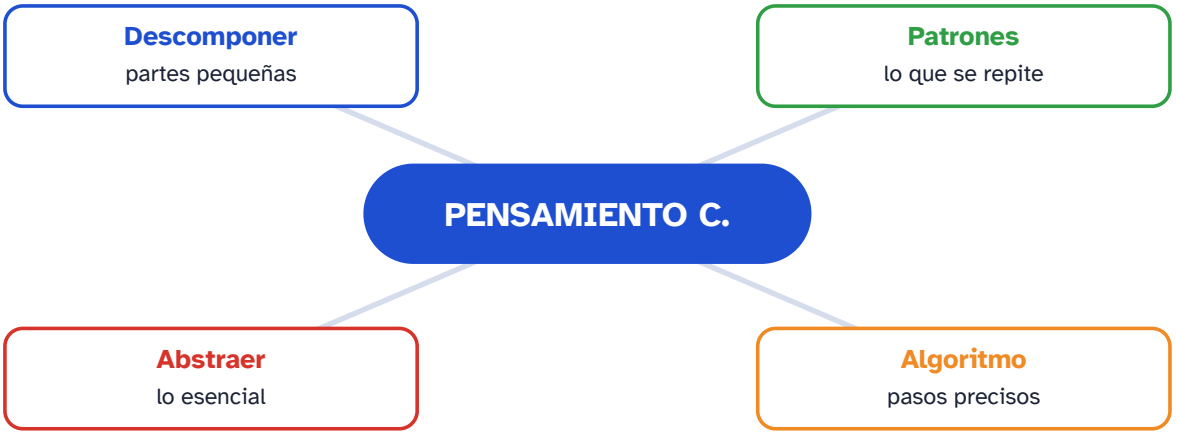
C) cambiar muchas cosas

D) ignorarlo

Explícalo con tus palabras

Incluye: Qué es el pensamiento computacional y por qué no es lo mismo que programar; Los cuatro pilares con un ejemplo de tu vida; Qué características debe tener un algoritmo; Cómo depurar un error.

Resumen



Glosario

Pensamiento computacional	Forma de resolver problemas para que una persona o máquina pueda ejecutar la solución.
Descomposición	Dividir un problema en partes.
Patrón	Algo que se repite.
Abstracción	Quedarse con lo esencial.
Generalización	Extender una solución a casos parecidos.
Algoritmo	Instrucciones finitas, ordenadas y precisas.
Depurar	Buscar y corregir errores.
Bug	Error de un programa.
Píxel	Punto de una imagen digital.
RGB	Modelo de color con rojo, verde y azul.
Grafo	Puntos unidos por líneas, a veces con pesos.
Eficiencia	Recursos que necesita un algoritmo.
Búsqueda binaria	Buscar en una lista ordenada descartando mitades.
Programación en parejas	Dos personas programan juntas con papeles alternos.
Aprendizaje automático	IA que aprende a partir de ejemplos.
Sesgo	Error sistemático por datos poco representativos.

Tarjetas de repaso

Recórtalas: pregunta delante, respuesta detrás.

Pensamiento computacional

Resolver problemas de forma que una persona o máquina pueda ejecutar la solución.

Descomposición

Dividir un problema en subproblemas.

Patrón

Algo que se repite y permite reutilizar soluciones.

Abstracción

Quedarse con lo importante y descartar lo irrelevante.

Algoritmo

Secuencia finita, ordenada y precisa de instrucciones.

Bug

Error en un programa.

Depurar

Buscar y corregir errores.

Actividad desenchufada

Practicar el pensamiento computacional sin ordenador.

Generalización

Hacer que una solución sirva para muchos casos.

RGB

Color de un píxel: rojo, verde y azul de 0 a 255.

Eficiencia

Pasos y memoria que necesita un algoritmo.

Aprendizaje automático

La máquina aprende de ejemplos.

Soluciones

1. **B)** Primer pilar.
2. **C)** Como el plano del metro.
3. **A)** Lleva a bucles y funciones.
4. **C)** Para que una máquina lo ejecute.
5. **B)** Falta precisión.
6. **A)** Debugging.
7. **B)** Investigadora estadounidense.
8. **C)** Patrón convertido en bucle.
9. **A)** Papel, juego, movimiento.
10. **B)** Así sabes qué lo arregló.
11. **A)** Se usan parámetros.
12. **B)** Una vuelta completa repartida.
13. **B)** Sin luz.
14. **A)** Cuadrícula de puntos.
15. **B)** Base del GPS.
16. **B)** Descarta la mitad cada vez.
17. **B)** Detecta errores y piensa el siguiente paso.
18. **B)** Muchos ejemplos etiquetados.
19. **A)** Repite los sesgos de los datos.
20. **B)** Que ocurra siempre.

Soluciones de los ejercicios

1. Descomposición (dividir), patrones (lo que se repite), abstracción (lo esencial) y algoritmos (pasos ordenados).
2. Por ejemplo: destino, transporte, presupuesto, autorizaciones, comida, horario.
3. Se suma 4 cada vez: 19.
4. Necesitas: colores, orden y duración de cada luz. Puedes ignorar: el material, la altura o la marca.
5. Por ejemplo: «calienta 200 ml de leche en el microondas a 800 W durante 60 segundos».
6. 360° / 3 = 120°.
7. Probar con casos extremos.
8. Porque elimina distancias y curvas reales y muestra solo estaciones, líneas y conexiones, que es lo que necesita el viajero.
9. Ejemplo: coger cepillo, poner pasta, cepillar 2 minutos, escupir, enjuagar, guardar cepillo.
10. Respuesta abierta; por ejemplo, una coreografía con un paso que se repite N veces.
11. Convertir la solución de un caso en una que sirva para todos los parecidos. Ejemplo: de dibujar un cuadrado a dibujar cualquier polígono de n lados.
12. Que sea correcto, eficiente (pocos pasos y memoria) y claro.
13. El conductor escribe el código; el navegante revisa, propone el siguiente paso y detecta errores. Se intercambian los papeles.
14. Con tres números de 0 a 255 que indican la cantidad de rojo, verde y azul.
15. Al programar, la persona escribe las reglas paso a paso; al entrenar, el ordenador construye un modelo a partir de muchos ejemplos.
16. Mochila → ropa (chaqueta, gorra), comida (agua, bocadillo, fruta), seguridad (botiquín, crema solar), documentos (tarjeta sanitaria, autorización).
17. Repetir n – 1 veces: avanzar; girar; repetir n – 1 veces: avanzar. El tamaño n es el parámetro.
18. Blanco: (255, 255, 255). Negro: (0, 0, 0). Gris medio: unos (128, 128, 128).
19. 100 · 100 = 10.000 píxeles · 3 = 30.000 bytes ≈ 30 KB.
20. Reproducir: buscar qué acción hace que baje de 0 (dos golpes a la vez). Aislar: el script de choque resta sin comprobar. Corregir: restar solo si vidas > 0. Probar: repetir el caso y los demás.
21. Unas 10 (2^{10} = 1.024) y unas 20 (2^{20} ≈ 1 millón).
22. Puntos: casa, cruce A, cruce B, cruce C, instituto. Líneas con su distancia: casa–A 200 m, A–B 150 m, A–C 300 m, B–instituto 250 m, C–instituto 100 m. El camino más corto se busca sumando pesos: casa–A–B–instituto = 600 m; casa–A–C–instituto = 600 m.
23. Porque casi ningún problema complejo se resuelve al primer intento: hay que probar, equivocarse, depurar y volver a intentarlo.
24. Un error sistemático debido a datos de entrenamiento poco variados. Ejemplo: un reconocedor de caras entrenado casi solo con personas de piel clara funciona peor con otras personas.

25. Guardar: líneas, paradas, horas de paso, tiempo entre paradas. Ignorar: color del autobús, nombre del conductor, modelo del vehículo.
26. Mientras queden pedidos: calcular la distancia a cada uno; ir al más cercano; marcarlo como entregado. Volver al almacén. No siempre es óptima: es una aproximación rápida (heurística) que suele dar buenas rutas, pero a veces una elección local lleva a un recorrido final largo.
27. Recorrerla una vez: n comparaciones. Ordenar requiere muchas más (del orden de $n \cdot \log n$ o n^2). Para el máximo basta un recorrido; ordenar hace trabajo innecesario.
28. Descomposición: recoger datos, entrenar, probar, conectar a Scratch. Patrones: muchas fotos de cada gesto. Abstracción: solo la mano, con fondo neutro. Algoritmo: si el modelo dice «gesto 1» → mover a la derecha… Probar con otras personas y depurar añadiendo ejemplos variados.
29. 1.920 · 1.080 = 2.073.600 píxeles · 3 ≈ 6,2 MB. Se comprimen (JPG, PNG) porque ocuparían mucho espacio y tardarían en enviarse; la compresión aprovecha patrones de la imagen.
30. Descomponer cada asignatura en temas; detectar patrones (todas necesitan resumen y ejercicios); abstraer (priorizar lo que más cuenta y lo que peor se lleva); algoritmo: calendario con bloques diarios, repaso espaciado y simulacro; evaluar y ajustar cada día.